

Thuật toán trên luồng dữ liệu

(Streaming & Sketching Algorithms)

Nhập môn: các bài toán và thuật toán cơ bản

Bài giảng chuyên đề

Biên soạn theo J. Nelson, *Sketching Algorithms* (Fall 2020),
Chương 2 và Chương 4

September 16, 2026

Nội dung

- 1 Mô hình streaming và sketching
- 2 Đếm xấp xỉ: thuật toán Morris
- 3 Đếm phần tử phân biệt (F_0)
- 4 Bài toán phân vị (quantiles)
- 5 Sketch tuyến tính (linear sketching)
- 6 Heavy hitters: CountMin và CountSketch
- 7 Sketch đồ thị: AGM
- 8 Ước lượng chuẩn: AMS và p -ổn định
- 9 Tổng kết

- Dữ liệu đến với tốc độ cao và khối lượng vượt xa bộ nhớ khả dụng:
 - bộ định tuyến Internet theo dõi lưu lượng mạng;
 - máy tìm kiếm theo dõi luồng truy vấn;
 - nhật ký giao dịch, cảm biến, đồ thị mạng xã hội biến thiên theo thời gian.
- Yêu cầu điển hình: *một lượt duyệt* (one pass), bộ nhớ nhỏ hơn hẳn kích thước dữ liệu, thời gian xử lý mỗi bản ghi phải nhỏ.
- Hệ quả: không thể tính chính xác; ta chấp nhận

xấp xỉ $(1 \pm \varepsilon)$ và xác suất thất bại δ .

- Câu hỏi trung tâm của môn học: *với bộ nhớ bao nhiêu thì tính được (xấp xỉ) đại lượng f ?*

Sketch là gì?

Định nghĩa (Sketch)

Một *sketch* của tập dữ liệu X đối với hàm f là một bản nén $C(X)$ sao cho từ $C(X)$ ta tính được (hoặc tính xấp xỉ được) giá trị $f(X)$.

- Tổng quát hơn: với hai tập X, Y ta muốn tính $f(X, Y)$ chỉ từ $C(X)$ và $C(Y)$.
- Ứng dụng phân tán: hai máy chủ trên mạng chỉ cần trao đổi sketch thay vì toàn bộ dữ liệu $\Rightarrow$ giảm băng thông.
- Ví dụ tầm thường: $f(X, Y) = \sum_{z \in X \cup Y} z$ với X, Y rời nhau; sketch = tổng các phần tử.
- Trong mô hình streaming, ta muốn *duy trì* $C(X)$ tại chỗ (on the fly) khi X được cập nhật.

Mô hình hình thức

Xét vector $x \in \mathbb{R}^n$ khởi tạo bằng 0. Luồng dữ liệu là dãy các cập nhật (i, Δ) , mỗi cập nhật thực hiện

$$x_i \leftarrow x_i + \Delta.$$

Ba mô hình được nghiên cứu phổ biến:

- ➊ **Chỉ chèn** (insertion-only): mọi $\Delta = 1$. Khi đó x là biểu đồ tần suất của luồng.
- ➋ **Turnstile chặt** (strict turnstile): Δ có thể âm, nhưng luôn có $x_i \geq 0$ tại mọi thời điểm.
- ➌ **Turnstile tổng quát** (general turnstile): không ràng buộc; x_i có thể âm.

Nhận xét

Mô hình turnstile chặt xuất hiện tự nhiên trong luồng đồ thị: $n = \binom{|V|}{2}$ và x_e là bội của cạnh e ; cạnh có thể bị chèn/xoá nhưng bội không bao giờ âm.

Các đại lượng cần đo

Với một thuật toán streaming, ta quan tâm:

- **Không gian** (space): số bit hoặc số từ máy (word) lưu trong bộ nhớ. Đây là tiêu chí chính.
- **Thời gian cập nhật** (update time): chi phí xử lý một cập nhật (i, Δ) .
- **Thời gian truy vấn** (query time).
- **Chất lượng**: sai số nhân $(1 \pm \varepsilon)$ hay sai số cộng $\pm \varepsilon n$; xác suất thất bại δ .
- **Tính hợp nhất** (mergeability): từ hai sketch của D_1, D_2 có dựng được sketch của $D_1 \sqcup D_2$ mà không suy giảm chất lượng?

Quy ước: $[n] := \{1, 2, \dots, n\}$; $\log$ là logarit cơ số 2 trừ khi nói rõ.

Ôn tập xác suất: các bất đẳng thức tập trung

Bổ đề (Markov)

Nếu $X \geq 0$ thì $\forall \lambda > 0, \mathbb{P}(X > \lambda) < \mathbb{E}X/\lambda$.

Bổ đề (Chebyshev)

$\forall \lambda > 0, \mathbb{P}(|X - \mathbb{E}X| > \lambda) < \text{Var}[X]/\lambda^2$.

Định lý (Chernoff)

Cho $X_1, \dots, X_n$ độc lập, $X_i \in [0, 1]$, $X = \sum_i X_i$, $\mu = \mathbb{E}X$. Khi đó

$$\mathbb{P}(|X - \mu| > \lambda\mu) < 2e^{-\lambda^2\mu/3}, \quad 0 < \lambda \leq 1.$$

Định lý (Hoeffding)

Cho $X_1, \dots, X_n$ độc lập cùng phân phối Bernoulli(p). Khi đó với mọi $\varepsilon \in (0, 1)$,

$$\mathbb{P}\left(\sum_{i=1}^n X_i > (p + \varepsilon)n\right) < e^{-2\varepsilon^2 n}, \quad \mathbb{P}\left(\sum_{i=1}^n X_i < (p - \varepsilon)n\right) < e^{-2\varepsilon^2 n}.$$

Ôn tập xác suất: bất đẳng thức Khintchine

Định lý (Khintchine)

Cho $\sigma_1, \dots, \sigma_n$ là các biến Rademacher độc lập (phân phối đều trên $\{-1, 1\}$) và $x \in \mathbb{R}^n$ cố định. Khi đó

$$\forall \lambda > 0, \quad \mathbb{P}(|\langle \sigma, x \rangle| > \lambda) \leq 2e^{-\lambda^2 / (2\|x\|_2^2)}.$$

Ý tưởng chứng minh.

Chặn hàm sinh moment: $\mathbb{E}e^{t\langle \sigma, x \rangle} = \prod_i \frac{1}{2}(e^{-tx_i} + e^{tx_i}) \leq \prod_i e^{t^2 x_i^2 / 2} = e^{t^2 \|x\|_2^2 / 2}$ (so sánh khai triển Taylor theo từng hạng tử), rồi áp dụng Markov cho $e^{t\langle \sigma, x \rangle}$ và tối ưu $t = \lambda / \|x\|_2^2$. □

- Ý nghĩa: tổ hợp dấu ngẫu nhiên $\sum_i \sigma_i x_i$ là biến dưới Gauss với phương sai $\|x\|_2^2$.
- Bất đẳng thức này là hạt nhân của phân tích sketch KLL cho bài toán phân vị (Mục 39).

Khuôn mẫu chung: “trung vị của trung bình”

Hầu hết các thuật toán trong bài giảng theo đúng khuôn mẫu sau.

- 1 **Thiết kế một ước lượng không chệch** $\tilde{Z}$ của đại lượng cần tính, duy trì được trong bộ nhớ nhỏ, với phương sai bị chặn.
- 2 **Lấy trung bình** $s = \Theta(1/\epsilon^2)$ bản sao độc lập. Phương sai giảm đi s lần; theo Chebyshev, xác suất thất bại hạ xuống hằng số (chẳng hạn $1/3$).
- 3 **Lấy trung vị** của $t = \Theta(\log(1/\delta))$ khối như trên. Theo Hoeffding, xác suất trung vị sai giảm xuống δ .

Chi phí: nhân không gian với thừa số $\Theta(\epsilon^{-2} \log(1/\delta))$.

Nhận xét

Vì sao cần trung vị chứ không tăng s ? Vì phụ thuộc theo Chebyshev là $1/\delta$, còn khuếch đại bằng trung vị cho $\log(1/\delta)$ — tốt hơn theo hàm mũ.

Bài toán đếm xấp xỉ

Bài toán (Morris, 1978)

Duy trì một số nguyên n với hai thao tác:

- `update()`: tăng n thêm 1;
- `query()`: xuất ra (ước lượng của) n .

- Lời giải tầm thường: một bộ đếm, tốn $O(\log n)$ bit.
- Có thể chứng minh rằng giải *chính xác* bài toán này đòi hỏi $\Omega(\log n)$ bit.
- Do đó ta hạ yêu cầu: xuất $\tilde{n}$ thỏa mãn

$$\mathbb{P}(|\tilde{n} - n| > \varepsilon n) < \delta,$$

với $\varepsilon, \delta \in (0, 1)$ cho trước từ đầu luồng.

- **Mục tiêu:** không gian $O(\log \log n)$ — tốt hơn theo hàm mũ.

Thuật toán Morris

Morris

- ➊ Khởi tạo $X \leftarrow 0$.
 - ➋ Với mỗi `update`: tăng X lên 1 với xác suất $\frac{1}{2^X}$.
 - ➌ Với `query`: xuất ra $\tilde{n} = 2^X - 1$.
- Trực giác: X cố gắng lưu giá trị $\approx \log_2 n$. Khi bộ đếm đã lớn, ta tăng nó ngày càng hiếm.
 - Do chỉ lưu $X \approx \log_2 n$, bộ nhớ là $O(\log \log n)$ bit.
 - Ký hiệu: X_n là giá trị của X sau n lần `update`.

Phân tích: moment bậc một

Khẳng định

$$\mathbb{E}2^{X_n} = n + 1.$$

Chứng minh.

Quy nạp theo n . Trường hợp cơ sở $n = 0$ hiển nhiên. Bước quy nạp:

$$\begin{aligned}\mathbb{E}2^{X_{n+1}} &= \sum_{j=0}^{\infty} \mathbb{P}(X_n = j) \cdot \mathbb{E}(2^{X_{n+1}} \mid X_n = j) \\ &= \sum_{j=0}^{\infty} \mathbb{P}(X_n = j) \cdot \left(2^j \left(1 - \frac{1}{2^j}\right) + \frac{1}{2^j} \cdot 2^{j+1}\right) \\ &= \sum_{j=0}^{\infty} \mathbb{P}(X_n = j)2^j + \sum_j \mathbb{P}(X_n = j) = \mathbb{E}2^{X_n} + 1 = (n + 1) + 1.\end{aligned}$$

- Đây chính là lý do xuất $\tilde{n} = 2^X - 1$: đó là ước lượng *không chệch* của n .

Phân tích: moment bậc hai

Khẳng định

$$\mathbb{E}2^{2X_n} = \frac{3}{2}n^2 + \frac{3}{2}n + 1.$$

Chứng minh bằng quy nạp, tương tự khẳng định trước. Từ đó

$$\mathbb{E}(\tilde{n} - n)^2 = \mathbb{E}(2^{X_n} - 1 - n)^2 = \frac{1}{2}n^2 - \frac{1}{2}n - 1 < \frac{1}{2}n^2.$$

Áp dụng Chebyshev:

$$\mathbb{P}(|\tilde{n} - n| > \varepsilon n) < \frac{1}{\varepsilon^2 n^2} \cdot \frac{n^2}{2} = \frac{1}{2\varepsilon^2}.$$

Nhận xét

Chặn này *vô nghĩa* khi $\varepsilon < 1$: vế phải chỉ nhỏ hơn $1/2$ khi $\varepsilon \geq 1$. Thuật toán cơ sở có phương sai quá lớn — cần khuếch đại.

Morris+: lấy trung bình

Morris+

Chạy s bản sao độc lập của Morris, thu được $\tilde{n}_1, \dots, \tilde{n}_s$, và xuất ra

$$\tilde{n} = \frac{1}{s} \sum_{i=1}^s \tilde{n}_i.$$

- Mỗi $\tilde{n}_i$ không chệch $\Rightarrow$ trung bình cũng không chệch.
- Phương sai của tổng các biến độc lập cộng lại, và nhân với $1/s$ làm phương sai giảm s^2 lần; tổng hợp lại phương sai giảm s lần:

$$\mathbb{P}(|\tilde{n} - n| > \varepsilon n) < \frac{1}{2s\varepsilon^2} < \delta \quad \text{khi } s > \frac{1}{2\varepsilon^2\delta} = \Theta\left(\frac{1}{\varepsilon^2\delta}\right).$$

- Phụ thuộc $1/\delta$ vẫn còn đắt. Ta khắc phục bằng bước tiếp theo.

Morris++: lấy trung vị

Morris++

Chạy t bản sao độc lập của Morris+, mỗi bản với xác suất thất bại $1/3$ (tức $s = \Theta(1/\varepsilon^2)$), rồi xuất ra *trung vị* của t ước lượng.

Phân tích.

Đặt $Y_i = 1$ nếu bản sao thứ i của Morris+ thành công, $Y_i = 0$ nếu ngược lại. Khi đó $\mathbb{E} \sum_i Y_i \geq 2t/3$. Trung vị sai chỉ khi nhiều nhất một nửa số bản sao thành công, tức là $\sum_i Y_i$ lệch khỏi kỳ vọng ít nhất $t/6$. Theo Hoeffding,

$$\mathbb{P}\left(\sum_i Y_i \leq \frac{t}{2}\right) \leq \mathbb{P}\left(\sum_i Y_i - \mathbb{E} \sum_i Y_i < -\frac{t}{6}\right) < e^{-2(1/6)^2 t} \leq \delta$$

với $t \geq \lceil 18 \ln(1/\delta) \rceil$.



Độ phức tạp không gian của Morris++

Tổng cộng ta chạy $st = \Theta(\varepsilon^{-2} \log(1/\delta))$ bản sao Morris cơ sở.

- Khi một bộ đếm X đạt tới $\log(stn/\delta)$, xác suất nó được tăng tại một bước bất kỳ là $\leq \delta/(nst)$; qua n bước, xác suất nó tăng thêm lần nữa là $\leq \delta/(st)$.
- Hợp biến cố (union bound): với xác suất $1 - \delta$, không bộ đếm nào vượt quá $\log(stn/\delta)$, tức cần $O(\log \log(stn/\delta))$ bit mỗi bộ đếm.

Định lý

Morris++ cho xấp xỉ $(1 \pm \varepsilon)$ với xác suất $1 - \delta$ dùng

$$O(\varepsilon^{-2} \log(1/\delta) \log \log(n/(\varepsilon\delta))) \text{ bit.}$$

Với ε, δ hằng số, không gian là $O(\log \log n)$.

So sánh: bộ đếm thông thường tốn $\Theta(\log n)$ bit. Cải thiện theo hàm mũ.

Tính chĩnh: cơ số $1 + \alpha$

Nhận xét (Morris)

Thay xác suất tăng $1/2^X$ bằng $1/(1 + \alpha)^X$ và ước lượng

$$\tilde{n} := \frac{(1 + \alpha)^X - 1}{\alpha}.$$

- Trực giác: nếu tăng với xác suất 1.0^X ta được bộ đếm tất định — phương sai bằng 0 nhưng tốn bộ nhớ; nếu tăng với xác suất 0.5^X thì tiết kiệm bộ nhớ nhưng phương sai lớn. Tham số α nội suy giữa hai thái cực.
- Chặn phương sai rồi dùng Chebyshev cho thấy $\alpha = \Theta(\varepsilon^2 \delta)$ đủ để đạt xấp xỉ $(1 + \varepsilon)$ với xác suất $1 - \delta$, dùng

$$O(\log(1/\varepsilon) + \log \log n + \log(1/\delta)) \text{ bit.}$$

- Phân tích tinh hơn cho $O(\log(1/\varepsilon) + \log \log n + \log \log(1/\delta))$, và chặn này là *tối ưu* (Nelson–Yu, 2020).

Bài toán (Distinct elements)

Cho luồng $i_1, \dots, i_m \in [n]$. Xuất ra $t =$ số phần tử *phân biệt* xuất hiện trong luồng.

- Hai lời giải tầm thường:
 - mảng bit độ dài n (n bit);
 - lưu toàn bộ luồng ($m \lceil \log n \rceil$ bit).Tức chính xác được với $\min\{n, m \lceil \log n \rceil\}$ bit.
- Ta hạ yêu cầu xuống $\tilde{t}$ với $\mathbb{P}(|t - \tilde{t}| > \varepsilon t) < \delta$.
- Tên gọi F_0 : nếu $x \in \mathbb{R}^n$ là biểu đồ tần suất thì $F_p = \sum_i x_i^p$; số phần tử phân biệt là giới hạn của F_p khi $p \rightarrow 0$.
- Công trình đầu tiên: Flajolet–Martin (1985).

Thuật toán FM lý tưởng hoá

FM (idealized)

- 1 Chọn hàm ngẫu nhiên $h : [n] \rightarrow [0, 1]$.
- 2 Duy trì $X = \min_{i \in \text{luồng}} h(i)$.
- 3 Xuất ra $1/X - 1$.

- “Lý tưởng hoá” vì ta không thể lưu h thực sự ngẫu nhiên trong $o(n)$ bit: có n biến ngẫu nhiên độc lập $(h(i))_{i=1}^n$, và giá trị là số thực.
- Trực giác: X là cực tiểu của t biến độc lập cùng phân phối $\text{Unif}(0, 1)$. Ta tính kỳ vọng của cực tiểu theo t rồi *ngịch đảo* quan hệ đó.
- Chú ý tính chất then chốt: giá trị X *không phụ thuộc bội* — phần tử lặp lại không làm thay đổi cực tiểu. Đó là lý do FM đo được F_0 .

Phân tích FM: hai moment đầu

Khẳng định

$$\mathbb{E}X = \frac{1}{t+1}.$$

Chứng minh.

$$\mathbb{E}X = \int_0^\infty \mathbb{P}(X > \lambda) d\lambda = \int_0^\infty \prod_{r=1}^t \mathbb{P}(h(i_r) > \lambda) d\lambda = \int_0^1 (1-\lambda)^t d\lambda = \frac{1}{t+1}. \quad \square$$

Khẳng định

$$\mathbb{E}X^2 = \frac{2}{(t+1)(t+2)}.$$

Chứng minh.

$$\mathbb{E}X^2 = \int_0^1 \mathbb{P}(X > \sqrt{\lambda}) d\lambda = \int_0^1 (1-\sqrt{\lambda})^t d\lambda = 2 \int_0^1 u^t (1-u) du = \frac{2}{(t+1)(t+2)}, \text{ với phép thế } u = 1 - \sqrt{\lambda}. \quad \square$$

$$\text{Do đó } \text{Var}[X] = \frac{t}{(t+1)^2(t+2)} < (\mathbb{E}X)^2 = \frac{1}{(t+1)^2}.$$

FM+ và FM++

FM+

Chạy $s = \lceil 1/(\varepsilon^2 \eta) \rceil$ bản sao độc lập; đặt $Z = \frac{1}{s} \sum_i X_i$ và xuất ra $1/Z - 1$.

Ta có $\mathbb{E}Z = \frac{1}{t+1}$ và $\text{Var}[Z] < \frac{1}{s(t+1)^2}$, nên theo Chebyshev

$$\mathbb{P}\left(\left|Z - \frac{1}{t+1}\right| > \frac{\varepsilon}{t+1}\right) < \eta \quad \Rightarrow \quad \mathbb{P}\left(\left|\frac{1}{Z} - 1 - t\right| > O(\varepsilon)t\right) < \eta.$$

(Với $\frac{1}{(1 \pm \varepsilon)/(t+1)} - 1 = (1 \pm O(\varepsilon))(t+1) - 1 = (1 \pm O(\varepsilon))t$.)

FM++

Chạy $q = \lceil 18 \ln(1/\delta) \rceil$ bản sao FM+ với $\eta = 1/3$ và xuất ra trung vị. Theo Hoeffding, xác suất thất bại $< \delta$.

Không gian (bỏ qua h): $O(\varepsilon^{-2} \log(1/\delta))$ số thực.

Hàm băm độc lập bậc k

Định nghĩa

Họ $\mathcal{H}$ các hàm từ $[a]$ vào $[b]$ là *độc lập bậc k* (k -wise independent) nếu với mọi $j_1, \dots, j_k \in [b]$ và mọi $i_1, \dots, i_k \in [a]$ phân biệt,

$$\mathbb{P}_{h \in \mathcal{H}}(h(i_1) = j_1 \wedge \dots \wedge h(i_k) = j_k) = 1/b^k.$$

- Ví dụ 1: tập *mọi* hàm $[a] \rightarrow [b]$ độc lập bậc k với mọi k ; cần $a \log b$ bit để lưu — quá đắt.
- Ví dụ 2: $a = b = q$ với q là lũy thừa nguyên tố; $\mathcal{H}_{\text{poly}}$ là tập mọi đa thức bậc $\leq k - 1$ trên $\mathbb{F}_q$. Khi đó h lưu được trong $k \log a$ bit.

Khẳng định (Wegman–Carter)

$\mathcal{H}_{\text{poly}}$ độc lập bậc k .

Chứng minh.

Với $((i_r, j_r))_{r=1}^k$ cho trước, nội suy Lagrange cho đúng *một* đa thức bậc $\leq k - 1$ thoả mãn $h(i_r) = j_r$.
Vậy xác suất bằng $1/|\mathcal{H}_{\text{poly}}| = 1/q^k = 1/b^k$. □

Thuật toán KMV (*k* minimum values)

Ý tưởng: thay vì giữ giá trị băm nhỏ nhất, giữ k giá trị băm nhỏ nhất.

KMV (Bar-Yossef và cộng sự, 2002)

- 1 Chọn $h : [n] \rightarrow [M]$, $M = n^3$, từ họ độc lập *bậc hai*.
 - 2 Đặt $k = \lceil 24/\varepsilon^2 \rceil$; duy trì k giá trị băm nhỏ nhất đã gặp.
 - 3 Nếu thấy ít hơn k giá trị băm phân biệt: xuất ra đúng số giá trị đó.
 - 4 Ngược lại, gọi X là giá trị băm nhỏ thứ k ; xuất ra $\tilde{t} = kM/X$.
- Vì sao lấy thống kê thứ k ? Giá trị nhỏ nhất có phương sai lớn: chỉ cần một phần tử băm vào giá trị rất bé là ước lượng hỏng. Thống kê thứ tự thứ k bền vững hơn nhiều.
 - Vì sao $\tilde{t} = kM/X$? Với t giá trị băm rải đều trên $[0, 1]$, giá trị nhỏ thứ k vào khoảng $v \approx k/(t+1)$, nên $t \approx k/v - 1 \approx k/v$; rời rạc hoá $[0, 1]$ thành bội của $1/M$ cho $X = Mv$.
 - Không gian: $O(\varepsilon^{-2} \log n)$ bit (cộng $O(\log n)$ cho h).

Phân tích KMV (1/2): chặn trên

Ta chứng minh mỗi biến cố xấu xảy ra với xác suất $\leq 1/6$.

Biến cố $\tilde{t} > (1 + \varepsilon)t$. Vì $X = kM/\tilde{t}$, điều này chỉ xảy ra nếu ít nhất k chỉ số phân biệt bám vào giá trị nhỏ hơn $kM/((1 + \varepsilon)t)$. Đặt

$$Y_i = \mathbf{1}[\text{phần tử phân biệt thứ } i \text{ bám xuống dưới } kM/((1 + \varepsilon)t)], \quad Y = \sum_{i=1}^t Y_i.$$

Khi đó $\mathbb{E}Y_i < k/((1 + \varepsilon)t)$, suy ra $\mathbb{E}Y < k/(1 + \varepsilon)$, và do $\text{Var}[Y_i] < \mathbb{E}Y_i^2 = \mathbb{E}Y_i$ cùng tính độc lập từng đôi,

$$\text{Var}[Y] = \sum_i \text{Var}[Y_i] < \frac{k}{1 + \varepsilon}.$$

Theo Chebyshev,

$$\mathbb{P}(Y \geq k) \leq \mathbb{P}\left(|Y - \mathbb{E}Y| > k\left(1 - \frac{1}{1+\varepsilon}\right)\right) < \frac{k}{1 + \varepsilon} \cdot \frac{(1 + \varepsilon)^2}{k^2 \varepsilon^2} = \frac{1 + \varepsilon}{\varepsilon^2 k} < \frac{1}{6}.$$

Phân tích KMV (2/2): chặn dưới

Biến cố $\tilde{t} < (1 - \varepsilon)t$ chỉ xảy ra khi *không đủ* chỉ số băm xuống vùng nhỏ. Đặt

$$Z_i = \mathbf{1}[\text{phần tử phân biệt thứ } i \text{ băm xuống dưới } kM/((1 - \varepsilon)t)], \quad Z = \sum_i Z_i,$$

thì $\tilde{t} < (1 - \varepsilon)t$ kéo theo $Z < k$. Ta có (sai số $1/M$ do rời rạc hoá)

$$\frac{k}{(1 - \varepsilon)t} \geq \mathbb{E}Z_i > \frac{k}{(1 - \varepsilon)t} - \frac{1}{M} \geq \frac{(1 + \varepsilon)k}{t} - \frac{1}{M}.$$

Vì $\varepsilon > 1/\sqrt{n}$ và $t < n$ nên $1/M < \varepsilon k/(4t)$, do đó $\mathbb{E}Z_i > (1 + 3\varepsilon/4)k/t$, suy ra

$$\frac{k}{1 - \varepsilon} \geq \mathbb{E}Z > \left(1 + \frac{3\varepsilon}{4}\right)k, \quad \text{Var}[Z] \leq \mathbb{E}Z \leq \frac{k}{1 - \varepsilon}.$$

Theo Chebyshev,

$$\mathbb{P}(Z < k) < \mathbb{P}\left(|Z - \mathbb{E}Z| > \frac{3}{4}\varepsilon k\right) < \frac{k}{1 - \varepsilon} \cdot \frac{16}{9\varepsilon^2 k^2} < \frac{16}{9(1 - \varepsilon)} \cdot \frac{1}{\varepsilon^2 k} < \frac{1}{6}.$$

Hợp lại: xác suất thất bại $\leq 1/3$; khuếch đại bằng trung vị như thường lệ.

Cách tiếp cận khác: lấy mẫu hình học

Lấy mẫu hình học (geometric sampling) là kỹ thuật phổ biến trong thiết kế thuật toán streaming.

- Chọn $h : [n] \rightarrow \{0, 1, \dots, \log n\}$ từ họ độc lập bậc hai với

$$\mathbb{P}(h(i) = j) = 2^{-(j+1)} \quad (j < \log n), \quad \mathbb{P}(h(i) = \log n) = 1/n.$$

(Hiện thực: lấy $h' : [n] \rightarrow [n]$ độc lập từng đôi, đặt $h(i)$ là vị trí bit thấp nhất bằng 1 của $h'(i)$.)

- h chia luồng thành $\log n + 1$ luồng con $S_0, \dots, S_{\log n}$ với $S_r = \{i : h(i) = r\}$; mỗi S_r đưa vào một cấu trúc D_r .
- D_r giải bài toán hứa hẹn: nếu số phần tử phân biệt $\leq k$ thì xuất chính xác, ngược lại có thể thất bại tùy ý. Lời giải tất định: ghi lại k phần tử phân biệt đầu tiên, dùng $O(k \log n)$ bit.
- Kỳ vọng số phần tử phân biệt ở mức r là $t/2^r$.

Xấp xỉ sai số hằng số

Dùng $k_1 = 1$: mỗi D_r chỉ cần một bit đánh dấu “đã thấy phần tử nào chưa”. Xuất ra 2^ℓ với ℓ là chỉ số lớn nhất sao cho $D_\ell.\text{query}() \neq 0$. Đặt $L := \log_2 t$.

Bổ đề

$$\mathbb{P}(\ell > L + 4) \leq 1/8.$$

Chứng minh.

$$\mathbb{P}(\ell > L + 4) \leq \sum_{r=\lceil L+4 \rceil}^{\log n} \mathbb{P}(F_0(r) \geq 1) \leq \sum_{r \geq \lceil L+4 \rceil} \frac{t}{2^r} < \sum_{q \geq 4} 2^{-q} = \frac{1}{8}, \text{ dùng Markov và } \mathbb{E}_h F_0(r) = t/2^r. \quad \square$$

Xấp xỉ sai số hằng số (tiếp)

Bổ đề

$$\mathbb{P}(F_0(\lceil L - 4 \rceil) = 0) < 1/8.$$

Chứng minh.

Đặt $q = F_0(\lceil L - 4 \rceil)$, khi đó $\mathbb{E}q = t/2^{\lceil L - 4 \rceil}$ và $\text{Var}[q] < \mathbb{E}q$ do tính độc lập từng đôi. Chebyshev cho $\mathbb{P}(q = 0) \leq \text{Var}[q]/(\mathbb{E}q)^2 \leq 1/\mathbb{E}q < 1/8$. □

Hệ quả

$2^\ell \in [\frac{1}{16}t, 16t]$ với xác suất ít nhất $3/4$.

Tính chỉnh lên xấp xỉ $(1 + \varepsilon)$

Chạy song song thuật toán trên với $k_2 = \lceil 512/\varepsilon^2 + 68/\varepsilon \rceil$ và điều kiện hoá trên biến cố tốt $\hat{t} := 2^\ell \in [\frac{1}{16}t, 16t]$.

- Nếu $t \leq k_2$: trả về $D_0.\text{query}()$, đúng *chính xác*.
- Ngược lại, chọn $r^* = \lceil \log_2(\varepsilon^2 \hat{t}/32) \rceil$, khi đó $1/\varepsilon^2 < t/2^{r^*} \leq 512/\varepsilon^2$.
- Vì $\text{Var}_h[F_0(r^*)] \leq t/2^{r^*}$, Chebyshev cho

$$F_0(r^*) = t/2^{r^*} \pm 3\sqrt{512}/\varepsilon = (1 \pm 68\varepsilon) t/2^{r^*}$$

với xác suất $\geq 8/9$; nhân ngược với 2^{r^*} ta được xấp xỉ $(1 + O(\varepsilon))$.

Định lý

Tồn tại thuật toán cho xấp xỉ $(1 + \varepsilon)$ của F_0 với xác suất $1 - \delta$ dùng $O(\varepsilon^{-2} \log^2 n \log(1/\delta))$ bit.

Nhận xét

Tốn bộ nhớ hơn KMV, nhưng thời gian cập nhật và truy vấn là $O(1)$ trong mô hình word RAM, trong khi KMV hiện thực bằng heap cần $\Theta(\log(1/\varepsilon))$.

Tổng kết bài toán F_0

Thuật toán	Không gian	Ghi chú
Tầm thường	$\min\{n, m \log n\}$	chính xác
FM lý tưởng	$O(\varepsilon^{-2} \log(1/\delta))$ số thực	cần ngẫu nhiên hoàn hảo
KMV	$O(\varepsilon^{-2} \log n)$ bit	chỉ cần độc lập bậc hai
Lấy mẫu hình học	$O(\varepsilon^{-2} \log^2 n)$ bit	cập nhật/truy vấn $O(1)$
KNW10	$O(\varepsilon^{-2} + \log n)$ bit	tối ưu với δ hằng số
HyperLogLog	$O(\varepsilon^{-2} \log \log n)$	tốt nhất trên thực tế

- Chặn tối ưu tổng quát: $\Theta(\varepsilon^{-2} \log(1/\delta) + \log n)$ bit.
- Chặn dưới $\Omega(1/\varepsilon^2)$ (với $\varepsilon > 1/\sqrt{n}$) suy ra từ độ phức tạp truyền thông của bài toán GAPHAMMING, và $\Omega(\log n)$ từ lập luận nén.
- *Cả ngẫu nhiên hoá lẫn xấp xỉ đều cần thiết*: thuật toán tất định hoặc chính xác đều đòi $\Omega(n)$ bit.

Bài toán phân vị

Luồng gồm n phần tử so sánh được $y_1, \dots, y_n$ theo thứ tự tùy ý; ký hiệu $x_1 < x_2 < \dots < x_n$ là chúng sau khi sắp xếp. Cho trước $\varepsilon \in [0, 1)$ từ đầu luồng, ta cần hỗ trợ:

- $\text{rank}(x)$: nếu r thỏa $x_r < x \leq x_{r+1}$ (hạng thật của x) thì trả về $r \pm \varepsilon n$;
- $\text{select}(r)$: trả về phần tử bất kỳ có hạng $r \pm \varepsilon n$;
- $\text{quantile}(\phi)$: viết gọn của $\text{select}(\phi n)$.
- Ứng dụng điển hình: giám sát độ trễ hệ thống, truy vấn phân vị $\phi = 0.999$ để bảo đảm 99,9% yêu cầu được phục vụ nhanh.
- Giả thiết các y_i phân biệt không mất tính tổng quát: thay y_i bởi cặp (y_i, i) và dùng thứ tự từ điển.
- Khác với F_0 : ở đây sai số là *cộng* ($\pm \varepsilon n$), không phải nhân.

Bức tranh tổng thể các kết quả

Sketch dựa trên so sánh (comparison-based): bộ nhớ chia thành M_1 (tập con các phần tử đã thấy) và M_2 (chỉ lưu kết quả các phép so sánh); mọi quyết định chỉ dựa trên kết quả so sánh.

- **Tất định, không dựa trên so sánh:** nếu giá trị nằm trong $\{0, \dots, U - 1\}$, sketch **q-digest** dùng $O(\varepsilon^{-1} \log U)$ từ.
- **Tất định, dựa trên so sánh:** sketch **GK** dùng $O(\varepsilon^{-1} \log(\varepsilon n))$ từ, và đây là tối ưu trong lớp này (Cormode–Vesely, 2020).
- **Ngẫu nhiên, dựa trên so sánh:** sketch **KLL** dùng $O(\varepsilon^{-1} \log \log(1/\delta))$ từ, đúng với xác suất $1 - \delta$ cho mỗi truy vấn cố định; bảo đảm “mọi phân vị” cần $O(\varepsilon^{-1} \log \log(1/(\varepsilon \delta)))$ từ. Chặn này tối ưu tới thừa số hằng.

Một số bài toán mở

q-digest có tối ưu không? Có thể vượt KLL bằng cách *không* dựa trên so sánh? Có cách phân tích đơn giản hơn cho GK?

Giả sử giá trị thuộc $\{0, \dots, U - 1\}$, U là lũy thừa của 2, và đặt $L := \log_2 U$.

- Tưởng tượng một cây nhị phân hoàn chỉnh mà các lá tương ứng với $0, 1, \dots, U - 1$. Mỗi đỉnh v có bộ đếm $c[v]$, đại diện cho khoảng giá trị từ lá trái nhất đến lá phải nhất trong cây con của nó.
- Cấu trúc lưu tên mọi v với $c[v] \neq 0$ cùng giá trị $c[v]$.
- Lý tưởng nhất: $c[v] = 0$ ở mọi đỉnh trong, và $c[v]$ bằng số lần xuất hiện của v ở mọi lá — tức là lưu nguyên biểu đồ tần suất, trả lời chính xác mọi truy vấn nhưng quá tốn bộ nhớ.
- Giải pháp: thỉnh thoảng *gộp* hai đỉnh anh em vào đỉnh cha (cộng dồn bộ đếm rồi xóa). Việc này tiết kiệm bộ nhớ nhưng làm mất độ chính xác: với đếm lưu tại đỉnh trong, ta không biết chính xác giá trị nằm ở đâu trong cây con.

Ký hiệu: $s[v]$ là đỉnh anh em, $p[v]$ là cha, $\text{left}[v]$, $\text{right}[v]$ là con trái/phải.

q-digest: mã giả

khởi tạo

- 1 $n \leftarrow 0$
- 2 $S \leftarrow \emptyset$ // các cặp $(v, c[v])$ với $c[v] \neq 0$
- 3 $K := \lceil 6L/\varepsilon \rceil$

insert(i)

- 1 $n \leftarrow n + 1$
- 2 nếu lá i có trong S : $c[i] \leftarrow c[i] + 1$
- 3 ngược lại: $S \leftarrow S \cup \{(i, 1)\}$
- 4 nếu $|S| > K$: gọi `compress()`

compress()

- 1 với $\ell = L, L - 1, \dots, 1$:
 - 2 với mỗi đỉnh $v \in S$ ở mức ℓ :
 - 3 nếu $c[p[v]] + c[v] + c[s[v]] \leq \varepsilon n / L$:
 - 4 $c[p[v]] \leftarrow c[p[v]] + c[v] + c[s[v]]$
 - 5 $c[v] \leftarrow 0$; $c[s[v]] \leftarrow 0$
 - 6 cập nhật S tương ứng
- `rank( $x$ )`: đi theo đường từ gốc tới lá x , cộng dồn $c[]$ của mọi tổ tiên và của mọi cây con trái rẽ nhánh.
 - `select( $r$ )`: tìm kiếm theo chiều sâu từ gốc, giữ tổng chạy A .

q-digest: tính đúng đắn và kích thước

Bổ đề

q-digest trả lời $rank$ và $select$ với sai số $\pm \epsilon n$.

Chứng minh.

Quan sát then chốt: mỗi phần tử của luồng đóng góp vào $c[]$ của *đúng một* đỉnh — lá của nó hoặc một tổ tiên. Với $rank(x)$: mọi $z \leq x$ hoặc tăng bộ đếm của một tổ tiên của x , hoặc của một đỉnh trong cây con trái của tổ tiên nào đó, nên đều được đếm. Với $z > x$, nó chỉ bị đếm nhầm khi tăng bộ đếm của một tổ tiên của x . Nhưng x chỉ có L tổ tiên, mỗi tổ tiên có bộ đếm $\leq \epsilon n/L$ nhờ `compress`, nên sai số tổng cộng $\leq L \cdot (\epsilon n/L) = \epsilon n$. □

q-digest: chặn kích thước bộ nhớ

Bổ đề

Sau mỗi lần gọi *compress*, ta có $|S| \leq 3L/\varepsilon + 1$.

Chứng minh.

Sau nén, mọi $v \in S$ không phải gốc thoả $c[p[v]] + c[v] + c[s[v]] > \varepsilon n/L$. Lấy tổng theo các v đó:

$$3n \geq \sum_{\substack{v \in S \\ v \neq \text{gốc}}} (c[p[v]] + c[v] + c[s[v]]) > (|S| - 1)\varepsilon n/L,$$

vì mỗi phần tử đóng góp vào đúng một bộ đếm và mỗi $c[v]$ xuất hiện nhiều nhất ba lần trong tổng. □

Sketch MRL: bộ nén (compactor)

MRL (Manku–Rajagopalan–Lindsay, 1998) là tất định và *dựa trên so sánh* — không cần biết vũ trụ giá trị, nhưng cần biết trước độ dài luồng n .

Bộ nén (compactor) với tham số k chẵn

Lưu từ 0 đến k phần tử. Khi đầy, nó *nén*: sắp xếp $x_1 < \dots < x_k$ rồi xuất ra các phần tử ở vị trí lẻ $x_1, x_3, \dots, x_{k-1}$ sang bộ nén ở mức trên.

- MRL xây chuỗi các bộ nén $A_0, A_1, \dots, A_L$, mỗi bộ kích thước $k = \varepsilon^{-1} \lceil \log(\varepsilon n) \rceil + 1$ (làm tròn lên số chẵn), $L = \lceil \log(n/k) \rceil$.
- Phần tử ở mức j mang *trọng số* 2^j : nó đại diện cho 2^j phần tử của luồng gốc.
- $\text{rank}(x)$: cộng 2^j cho mỗi phần tử $z < x$ nằm trong A_j , với mọi j .

MRL: phân tích sai số

- Khi nén ở mức j , sai số sinh ra cho $\text{rank}(x)$ là $\eta_x 2^j$ với $\eta_x \in \{0, 1\}$ phụ thuộc *tính chẵn lẻ* của hạng của x trong nhóm được nén. Nếu hạng đó chẵn thì không sinh sai số nào.
- Vậy sai số cộng dồn là

$$E(x) := \sum_{j=0}^{L-1} \sum_{i=1}^{m_j} \eta_{x,j,i} 2^j,$$

với m_j là số lần nén ở mức j .

- Có nhiều nhất $n/2^j$ phần tử từng được chèn vào A_j , mỗi lần nén xóa k phần tử, nên $m_j \leq n/(k2^j)$. Do đó

$$E(x) \leq \sum_{j=0}^{L-1} \frac{n}{k2^j} 2^j = \frac{n}{k} \cdot L.$$

- Để $E(x) \leq \varepsilon n$, ta cần $L/k \leq \varepsilon$, tức $k \geq \varepsilon^{-1} \lceil \log(n/k) \rceil$ — đúng như lựa chọn ở trên.

Định lý

Sketch MRL dùng không gian $O(k \log(n/k)) = O(\varepsilon^{-1} \log^2(\varepsilon n))$.

Sketch KLL: ngẫu nhiên hoá bộ nén

Điểm xuất phát: thay vì luôn xuất các phần tử ở vị trí lẻ, bộ nén *chọn ngẫu nhiên đều* giữa vị trí lẻ và vị trí chẵn. Khi đó sai số trở thành

$$E(x) = \sum_{j=0}^{L-1} \sum_{i=1}^{m_j} \eta_{x,j,i} \sigma_{j,i} 2^j, \quad \eta \in \{0, 1\}, \sigma \in \{-1, 1\} \text{ ngẫu nhiên đều.}$$

Đây chính là dạng của bất đẳng thức Khintchine với vector có tọa độ $(\eta_{x,j,i} 2^j)_{j,i}$:

$$\|x\|_2^2 \leq \sum_{j=0}^{L-1} m_j 2^{2j} \leq \frac{n}{k} \cdot 2^L \leq 2 \left(\frac{n}{k} \right)^2.$$

Do đó $\mathbb{P}(|E(x)| > \varepsilon n) \leq 2e^{-\varepsilon^2 k^2 / 4}$, nhỏ hơn δ khi $k = \lceil \varepsilon^{-1} \sqrt{2 \log(1/\delta)} \rceil$.

Định lý

Tồn tại sketch ngẫu nhiên, dựa trên so sánh, cho phân vị với sai số cộng $\pm \varepsilon n$, xác suất thất bại δ , dùng $O(\varepsilon^{-1} \sqrt{\log(1/\delta)} \log(\varepsilon n / \sqrt{\log(1/\delta)}))$ từ, không cần biết trước n .

KLL: các bước tối ưu hoá

- ❶ **Kích thước giảm dần theo cấp số nhân.** Bộ nén cuối A_L có kích thước k ; A_{L-j} có kích thước $\max\{2, \approx (2/3)^j k\}$. Kết quả: thừa số $\log n$ trở thành *cộng* thay vì *nhân*:

$$O(\varepsilon^{-1} \sqrt{\log(1/\delta)} + \log(\varepsilon n / \sqrt{\log(1/\delta)})) \text{ từ.}$$

- ❷ **Thay các bộ nén đáy bằng một bộ lấy mẫu.** Chỉ $O(\log k)$ bộ nén trên cùng có kích thước > 2 ; toàn bộ T bộ nén đáy cùng nhau chỉ sinh ra một phần tử ngẫu nhiên đều trong 2^T phần tử — hiện thực bằng $O(1)$ từ. Không gian: $O(k)$.
- ❸ **Giữ s bộ nén trên cùng có kích thước k .** Tách sai số thành α (phần ngẫu nhiên, áp dụng Khintchine với $\|x\|_2^2 \leq 2(n/k)^2 2^{-s}$) và $\beta \leq ns/k$ (phần tất định). Chọn $s = \lceil \ln \ln(1/\delta) \rceil$ và $k = O(\varepsilon^{-1} \log \log(1/\delta))$ cho không gian $O(\varepsilon^{-1} (\log \log(1/\delta))^2)$.
- ❹ **Thay s bộ nén trên cùng bằng sketch GK** (tất định, tối ưu).

Định lý

Tồn tại sketch ngẫu nhiên, dựa trên so sánh, cho phân vị với sai số $\pm \varepsilon n$ và xác suất thất bại δ dùng $O(\varepsilon^{-1} \log \log(1/\delta))$ từ, không cần biết trước n .

Tính hợp nhất (mergeability)

Định nghĩa

Một thuật toán sketching là *hợp nhất hoàn toàn* nếu từ hai sketch S_1, S_2 của D_1, D_2 ta dựng được sketch của $D_1 \sqcup D_2$ mà không suy giảm sai số hay xác suất thất bại, và điều này đúng với cây hợp nhất tùy ý.

- MRL *không* hợp nhất hoàn toàn: tham số k được đặt theo n ngay từ đầu, nên hai sketch với $k_1 \neq k_2$ khó ghép lại.
- KLL (bỏ bước tối ưu cuối dùng GK) có tính hợp nhất hoàn toàn, với không gian $O(\varepsilon^{-1}(\log \log(1/\delta))^2)$.
- **Bài toán mở:** có tồn tại sketch phân vị tất định, dựa trên so sánh, dùng $o(n)$ bộ nhớ và hợp nhất hoàn toàn hay không? (Dự đoán: không.)
- Tính hợp nhất là yêu cầu thực tiễn cốt lõi khi sketch được tính song song trên nhiều máy rồi gộp lại.

Sketch tuyến tính

Kỹ thuật tổng quát: với $x \in \mathbb{R}^n$, lưu Πx trong bộ nhớ, với $\Pi \in \mathbb{R}^{m \times n}$, $m \ll n$.

- Nếu Π có *biểu diễn cô đọng* (lưu ít hơn mn ô nhớ, và có thuật toán bộ nhớ nhỏ tính $\Pi_{i,j}$ từ i, j) thì sketch nén biểu diễn của x từ n xuống m đơn vị bộ nhớ.
- **Tính tuyến tính làm cho cập nhật trở nên tầm thường:** cập nhật (i, Δ) cho $x \leftarrow x + \Delta e_i$, do đó

$$y = \Pi x \mapsto y + \Delta \cdot \Pi^{(i)},$$

với $\Pi^{(i)}$ là cột thứ i của Π . Cập nhật âm cũng xử lý được y hệt $\Rightarrow$ mô hình turnstile.

- Cũng cho ngay tính hợp nhất: $\Pi x + \Pi x' = \Pi(x + x')$, rất hợp với tính toán phân tán.

Nhận xét

Mọi thuật toán đã biết cho turnstile chặt và turnstile tổng quát đều là sketch tuyến tính. Hơn nữa, người ta chứng minh được rằng mọi thuật toán trong hai mô hình này đều *chuyển được* thành sketch tuyến tính, chỉ mất thừa số logarit về không gian — miễn là thuật toán phải đúng trên các luồng rất dài.

Truy vấn điểm và phần tử trội

Xét x được cập nhật trong mô hình turnstile, cùng tham số nguyên $k > 0$ cho trước.

Bài toán (ℓ_1 point query, tham số k)

Trả lời truy vấn $\text{query}(i)$, $i \in [n]$, bằng một giá trị trong khoảng $x_i \pm \|x\|_1 / k$.

Bài toán (ℓ_1 heavy hitters, tham số k)

Trả về tập $L \subset [n]$ sao cho

- 1 $|L| = O(k)$;
- 2 $|x_i| > \|x\|_1 / k \implies i \in L$.

- Các chỉ số ở điều kiện (2) gọi là *k-heavy hitters* (phần tử trội) hay *k-frequent*.
- Số chỉ số vượt quá $1/k$ tổng $\|x\|_1$ nhỏ hơn k , nên có nhiều nhất $k - 1$ phần tử trội. Ta chỉ đòi hỏi L không lớn hơn mức cần thiết quá một thừa số hằng.

Phiên bản “đuôi” và quy dẫn

Nhận xét (Bảo đảm đuôi)

Đặt $x_{\text{tail}(k)}$ là x sau khi triệt tiêu k tọa độ lớn nhất theo trị tuyệt đối. Bảo đảm *đuôi* yêu cầu sai số $\|x_{\text{tail}(k)}\|_1 / k$ thay vì $\|x\|_1 / k$ — mạnh hơn hẳn. Khi đó có nhiều nhất $2k - 1$ phần tử trội.

Bổ đề

Nếu có thuật toán $\mathcal{A}$ giải $(3k, \ell_1)$ -point query với xác suất thất bại $\leq \delta/n$ dùng S từ nhớ, thì có thuật toán $\mathcal{A}'$ giải (k, ℓ_1) -heavy hitters với xác suất thất bại $\leq \delta$ dùng $S + 3k$ từ.

Chứng minh.

$\mathcal{A}'$ dùng $\mathcal{A}$ xử lý luồng; khi truy vấn, nó duyệt mọi $i \in [n]$, truy vấn điểm từng chỉ số và giữ lại $4k$ chỉ số có giá trị lớn nhất. Điều kiện hoá trên biến cố mọi truy vấn điểm đều đúng (xác suất $\geq 1 - \delta$ theo hợp biến cố): mỗi chỉ số k -frequent có giá trị truy vấn $> \|x\|_1 / k - \|x\|_1 / (3k) = (2/3) \|x\|_1 / k$, trong khi mọi i với $|x_i| < \|x\|_1 / (3k)$ có giá trị $< (2/3) \|x\|_1 / k$. Vì có nhiều nhất $3k$ chỉ số với $|x_i| \geq \|x\|_1 / (3k)$, danh sách trả về chứa mọi chỉ số k -frequent. □

CountMin sketch

CountMin (Cormode–Muthukrishnan, 2005)

- 1 Lưu L hàm băm $h_1, \dots, h_L : [n] \rightarrow [B]$, mỗi hàm chọn độc lập từ họ độc lập bậc hai.
- 2 Lưu các bộ đếm $C_{a,b}$, $a \in [L]$, $b \in [B]$, với $B = 2k$ và $L = \lceil \log_2(1/\delta) \rceil$.
- 3 Với cập nhật (i, Δ) : cộng Δ vào mọi $C_{a, h_a(i)}$, $a = 1, \dots, L$.
- 4 Với $\text{query}(i)$: xuất ra $\min_{1 \leq a \leq L} C_{a, h_a(i)}$.

- Không gian: $O(L)$ để lưu các hạt giống hàm băm, cộng $O(BL)$ từ cho các bộ đếm; tổng cộng $O(BL) = O(k \log(1/\delta))$ từ.
- Đây là sketch tuyến tính: Π là ma trận 0/1 với đúng L số 1 trên mỗi cột.
- Trình bày dưới đây theo mô hình turnstile *chặt* (nên min mới hợp lý).

CountMin: phân tích

Bổ đề

$\text{CountMin.query}(i)$ trả về $x_i \pm \|x\|_1 / k$ với xác suất $\geq 1 - \delta$.

Chứng minh.

Cố định i . Đặt $Z_j = \mathbf{1}[h_r(j) = h_r(i)]$. Với mỗi $r \in [L]$,

$$C_{r, h_r(i)} = x_i + \sum_{j \neq i} x_j Z_j =: x_i + E.$$

Do tính độc lập bậc hai, $\mathbb{E}Z_j = 1/B$, nên

$$\mathbb{E}E = \sum_{j \neq i} |x_j| \cdot \frac{1}{B} \leq \frac{\|x\|_1}{2k}.$$

Theo Markov, $\mathbb{P}(E > \|x\|_1 / k) < 1/2$. Vì L hàng độc lập và mỗi hàng cho ước lượng thừa,

$$\mathbb{P}\left(\min_r C_{r, h_r(i)} > x_i + \|x\|_1 / k\right) < 1/2^L \leq \delta.$$



CountMin: hệ quả cho heavy hitters

Kết hợp bổ đề truy dẫn (point query $\rightarrow$ heavy hitters) với $\delta \leftarrow \delta/n$ và $k \leftarrow 3k$:

Định lý

Có thuật toán giải ℓ_1 k -heavy hitters trong mô hình turnstile chặt với xác suất thất bại δ , không gian $O(k \log(n/\delta))$, thời gian cập nhật $O(\log(n/\delta))$, thời gian truy vấn $O(n \log(n/\delta))$.

- Thời gian truy vấn $O(n \log(n/\delta))$ là điểm yếu: ta phải duyệt *toàn bộ* $[n]$. Mục sau khắc phục bằng thủ thuật dyadic.
- Sai số một chiều: vì mọi $C_{r, h_r(i)} \geq x_i$ trong turnstile chặt, cực tiểu luôn là *ước lượng thừa*; ta chỉ cần chặn phần thừa.

CountMin: bảo đảm đuôi và turnstile tổng quát

Bổ đề

CountMin với $B \geq 4k$ bảo đảm $query(i) = x_i \pm \|x_{tail(k)}\|_1 / k$ với xác suất $\geq 1 - \delta$.

Chứng minh.

Gọi $H \subset [n]$ là tập k chỉ số lớn nhất theo trị tuyệt đối, $T := [n] \setminus H$. Với mỗi r , tách

$$C_{r,h_r(i)} - x_i = \underbrace{\sum_{j \in H \setminus \{i\}} x_j Z_j}_{E_1} + \underbrace{\sum_{j \in T \setminus \{i\}} x_j Z_j}_{E_2}.$$

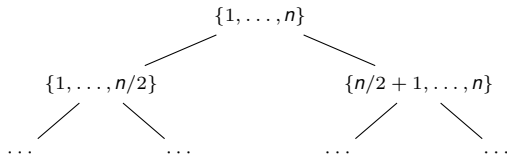
Biến cố F_r : $h_r(i) \notin h_r(H \setminus \{i\})$, xảy ra với xác suất $\geq 1 - |H|/B \geq 3/4$; khi đó $E_1 = 0$. Biến cố F'_r : $|E_2| \leq \|x_T\|_1 / k$, xảy ra với xác suất $\geq 3/4$ (lập luận như bổ đề trước). Vậy $\mathbb{P}(F_r \wedge F'_r) \geq 1/2$, và vì mỗi $C_{r,h_r(i)}$ đều $\geq x_i$, cực tiểu chỉ hỏng khi mọi hàng hỏng: xác suất $\leq 2^{-L} \leq \delta$. □

Nhận xét (Turnstile tổng quát)

Dùng *trung vị* thay cho cực tiểu, với B, L lớn hơn một thừa số hằng. Chẳng hạn $B = 3k$ cho $\mathbb{P}(|C_{r,h_r(i)} - x_i| > \|x\|_1 / k) < 1/3$, và Chernoff cho thấy trung vị đúng với xác suất $1 - \delta$ khi $L = C \log(1/\delta)$.

Tăng tốc truy vấn: thủ thuật dyadic

Thời gian truy vấn $O(n \log(n/\delta))$ là quá chậm. Khắc phục: xét cây nhị phân hoàn chỉnh với các lá tương ứng $[n]$.



- Mức j của cây tương ứng một vector $x^{(j)} \in \mathbb{R}^{2^j}$; giá trị tại một đỉnh là tổng giá trị hai con (lá thứ i mang giá trị x_i).
- Cập nhật (i, Δ) được đưa vào *mọi* tổ tiên của lá i . Ta lưu $1 + \log n$ sketch CountMin, mỗi mức một cái, mỗi cái giải bài toán $4k$ -heavy hitters với xác suất thất bại $\eta = \delta/(4k \log n)$.
- Tổng không gian: $O(k \log(1/\eta) \log n) = O(k \log \frac{k \log n}{\delta} \log n)$.

Thủ thuật dyadic: truy vấn và độ phức tạp

Quan sát then chốt (turnstile chặt): giá trị tại một tổ tiên luôn $\geq$ giá trị tại đỉnh con, và chuẩn ℓ_1 của vector ẩn tại mọi mức là *như nhau*. Vậy nếu i là phần tử trội ở mức lá thì mọi tổ tiên của i đều là phần tử trội ở mức tương ứng.

Truy vấn bằng tìm kiếm theo chiều sâu

Đi từ gốc xuống. Ở mỗi mức j , giữ danh sách L_j các phần tử trội. Với mỗi con của phần tử trong L_j , truy vấn điểm bằng sketch CountMin ở mức $j+1$; nếu giá trị $\geq (3/4) \|x\|_1 / k$ thì đưa vào L_{j+1} . Danh sách cuối cùng là L ở mức đáy.

- Mỗi $|L_j| \leq 2k$, nên ta truy vấn điểm nhiều nhất $4k$ con ở mức kế tiếp; tổng số truy vấn $\leq 4k \log n$. Hợp biến cố cho xác suất thất bại tổng cộng $\leq \delta$.
- Độ phức tạp:** không gian và thời gian truy vấn $O(k \log \frac{k \log n}{\delta} \log n)$ từ; thời gian cập nhật $O(\log n \log \frac{k \log n}{\delta})$.
- Thuật toán tốt nhất hiện biết cho ℓ_1 heavy hitters trong turnstile là *ExpanderSketch*: không gian $O(k \log(n/\delta))$ từ, cập nhật $O(\log(n/\delta))$, truy vấn $O(k \log(n/\delta) \text{ poly } \log n)$.

Vì sao bảo đảm ℓ_2 mạnh hơn ℓ_1 ?

Bổ đề

Với mọi $1 \leq p < q$:
$$\frac{\|x_{\text{tail}(2k)}\|_q}{k^{1/q}} \leq \frac{\|x_{\text{tail}(k)}\|_p}{k^{1/p}}.$$

Phác thảo.

Chia các chỉ số thành các khối $S_1, S_2, \dots$ kích thước k theo thứ tự giảm dần của $|x_i|$. Với mọi $i \in S_j$ ta có $|x_i| \leq (\|x_{S_{j-1}}\|_p^p / k)^{1/p}$, nên

$$\frac{\|x_{\text{tail}(2k)}\|_q}{k^{1/q}} \leq \frac{1}{k^{1/q}} \left(\sum_{j \geq 3} k \|x_{S_j}\|_\infty^q \right)^{1/q} \leq \frac{1}{k^{1/p}} \left(\sum_{j \geq 3} \|x_{S_{j-1}}\|_p^q \right)^{1/q} \leq \frac{\|x_{\text{tail}(k)}\|_p}{k^{1/p}},$$

bước cuối dùng $\|v\|_q \leq \|v\|_p$.



Ví dụ minh họa

Với $x = (\sqrt{n}, 1, 1, \dots, 1)$: chỉ số $i = 1$ là 1-heavy hitter theo ℓ_2 (thậm chí ở phiên bản không đuôi), trong khi theo ℓ_1 nó chỉ “trội” ở mức $O(1/\sqrt{n})$. Bảo đảm ℓ_2 phát hiện được nhiều phần tử trội hơn hẳn.

CountSketch (Charikar–Chen–Farach-Colton, 2004)

Ngoài $h_1, \dots, h_L : [n] \rightarrow [B]$, chọn thêm $\sigma_1, \dots, \sigma_L : [n] \rightarrow \{-1, 1\}$ độc lập từ họ độc lập bậc hai.

- Cập nhật (i, Δ) : cộng $\sigma_r(i)\Delta$ vào $C_{r, h_r(i)}$, $r = 1, \dots, L$.
- Truy vấn $\text{query}(i)$: xuất ra $\text{median}_{1 \leq r \leq L} \sigma_r(i) C_{r, h_r(i)}$.
- Bảo đảm: $\text{query}(i)$ trả về $x_i \pm \|x\|_2 / \sqrt{k}$ (ℓ_2 *point query*); phiên bản đuôi cho sai số $\|x_{\text{tail}(k)}\|_2 / \sqrt{k}$.
- ℓ_2 heavy hitters: trả về danh sách chứa mọi i với $|x_i| > \|x\|_2 / \sqrt{k}$.
- Khác biệt cốt yếu so với CountMin: các dấu ngẫu nhiên làm cho nhiều *triệt tiêu lẫn nhau*, biến sai số thiên lệch một chiều thành sai số có kỳ vọng bằng 0.

CountSketch: phân tích

Cố định i và hàng r ; đặt $\tilde{x}_i^{(r)} := \sigma_r(i) C_{r, h_r(i)}$. Khi đó

$$\tilde{x}_i^{(r)} = x_i + \sum_{j \neq i} \sigma_r(i) \sigma_r(j) x_j Z_j, \quad Z_j = \mathbf{1}[h_r(j) = h_r(i)].$$

- **Không chệch:** $\mathbb{E}[\sigma_r(i) \sigma_r(j)] = 0$ với $j \neq i$ do độc lập bậc hai, nên $\mathbb{E} \tilde{x}_i^{(r)} = x_i$.
- **Phương sai:** các số hạng chéo triệt tiêu, còn lại

$$\text{Var}[\tilde{x}_i^{(r)}] = \sum_{j \neq i} x_j^2 \mathbb{E} Z_j = \frac{1}{B} \sum_{j \neq i} x_j^2 \leq \frac{\|x\|_2^2}{B}.$$

- Chọn $B = 3k$; theo Chebyshev,

$$\mathbb{P}\left(\left|\tilde{x}_i^{(r)} - x_i\right| > \frac{\|x\|_2}{\sqrt{k}}\right) \leq \frac{\|x\|_2^2 / (3k)}{\|x\|_2^2 / k} = \frac{1}{3}.$$

- Lấy trung vị trên $L = O(\log(1/\delta))$ hàng độc lập, Chernoff cho xác suất thất bại $\leq \delta$.

Không gian: $O(k \log(1/\delta))$ từ — bằng CountMin, nhưng bảo đảm mạnh hơn.

Luồng đồ thị động

- Đồ thị $G = (V, E)$ với $|V| = N$; vector ẩn $x \in \mathbb{R}^{(N)}$ với x_e là bội của cạnh e . Cạnh được chèn và xoá $\Rightarrow$ mô hình turnstile chặt.
- **Mục tiêu:** xác định các thành phần liên thông (hoặc một rừng khung) dùng $\tilde{O}(N)$ bit — gọi là mô hình *semi-streaming*.
- Với luồng *chỉ chèn*, bài toán dễ: duy trì cấu trúc union-find, $O(N \log N)$ bit.
- Với xoá cạnh, union-find sụp đổ: một cạnh đã dùng để hợp nhất có thể bị xoá sau đó. Đây chính là chỗ sketch tuyến tính phát huy tác dụng.

Ba viên gạch

- 1 Khôi phục 1-thừa (1-sparse recovery);
- 2 `SupportFind`: tìm một phần tử bất kỳ trong giá của một vector khác 0;
- 3 Sketch AGM: dùng `SupportFind` bên trong thuật toán Borůvka.

Khôi phục 1-thừa

Bài toán

Cho vector x được cập nhật trong mô hình turnstile, với lời hứa rằng x có đúng một tọa độ khác 0. Tìm chỉ số và giá trị đó.

Lời giải: ba bộ đếm tuyến tính

Duy trì

$$s_0 = \sum_i x_i, \quad s_1 = \sum_i i x_i, \quad s_2 = \sum_i x_i z^i \bmod p,$$

với p là số nguyên tố và z ngẫu nhiên trong $\mathbb{F}_p$.

- Nếu $x = v e_j$ thì $s_0 = v$ và $s_1 = jv$, nên $j = s_1/s_0$ và $v = s_0$.
- s_2 dùng để *kiểm chứng*: kiểm tra $s_2 \stackrel{?}{=} s_0 z^j$. Nếu x không 1-thừa, đẳng thức chỉ đúng với xác suất $\leq n/p$ (đa thức khác 0 bậc $\leq n$ có $\leq n$ nghiệm — bổ đề Schwartz–Zippel).
- Cả ba đại lượng đều là *hàm tuyến tính* của x , nên đây là một sketch tuyến tính, dùng $O(\log n + \log p)$ bit.

SupportFind và khôi phục k -thừa

Bài toán (SupportFind)

Cho $x \neq 0$ trong turnstile tổng quát, trả về một chỉ số $i \in \text{supp}(x)$ với xác suất $\geq 1 - \delta$.

Lời giải: lấy mẫu hình học + khôi phục 1-thừa

Với mỗi mức $j = 0, 1, \dots, \log n$, lấy mẫu mỗi toạ độ với xác suất 2^{-j} (dùng hàm băm độc lập từng đôi) và chạy một cấu trúc khôi phục 1-thừa trên vector con đó.

- Tại mức j^* với $2^{j^*} \approx |\text{supp}(x)|$, kỳ vọng số toạ độ sống sót là $\Theta(1)$; với xác suất hằng số, vector con đúng 1-thừa và cấu trúc trả về một chỉ số hợp lệ.
- Bộ kiểm chứng s_2 bảo đảm ta không chấp nhận đầu ra rác từ các mức khác.
- Không gian: $O(\log^2 n)$ bit mỗi bản sao; lặp lại $O(\log(1/\delta))$ lần.
- **Khôi phục k -thừa** suy ra bằng cách băm $[n]$ vào $O(k)$ xô rồi chạy khôi phục 1-thừa trong mỗi xô.

Sketch AGM: vector kề có dấu

Với mỗi đỉnh $i \in V$, định nghĩa $a_i \in \mathbb{R}^{\binom{N}{2}}$ bởi

$$a_i[\{u, v\}] = \begin{cases} +1, & i = u < v \text{ và } \{u, v\} \in E, \\ -1, & i = v > u \text{ và } \{u, v\} \in E, \\ 0, & \text{ngược lại.} \end{cases}$$

Mệnh đề (Tính chất then chốt)

Với mọi $S \subseteq V$,

$$\text{supp}\left(\sum_{i \in S} a_i\right) = E(S, V \setminus S),$$

tức là giá của tổng chính xác bằng tập cạnh cắt qua S .

Chứng minh.

Một cạnh $\{u, v\}$ với $u, v \in S$ đóng góp $+1$ từ a_u và -1 từ a_v , nên triệt tiêu. Cạnh với đúng một đầu mút trong S đóng góp ± 1 và tồn tại. Cạnh không có đầu mút nào trong S đóng góp 0. □

Sketch AGM: liên thông trong luồng động

Thuật toán (Ahn–Guha–McGregor, 2012)

- 1 Với mỗi đỉnh i , lưu $t = O(\log N)$ sketch `SupportFind` độc lập của a_i : $\Pi^{(1)}a_i, \dots, \Pi^{(t)}a_i$.
- 2 Chạy t vòng kiểu Borůvka. Ở vòng ℓ , mỗi siêu đỉnh S (thành phần hiện thời) dùng

$$\Pi^{(\ell)}\left(\sum_{i \in S} a_i\right) = \sum_{i \in S} \Pi^{(\ell)}a_i$$

— tính được nhờ *tính tuyến tính* — để tìm một cạnh đi ra khỏi S .

- 3 Hợp nhất các siêu đỉnh dọc theo các cạnh tìm được.

- Mỗi vòng giảm ít nhất một nửa số thành phần $\Rightarrow O(\log N)$ vòng là đủ.
- Vì sao cần sketch độc lập cho mỗi vòng? Vì siêu đỉnh ở vòng ℓ *phụ thuộc* vào kết quả các vòng trước; dùng lại cùng ngẫu nhiên sẽ phá vỡ tính độc lập (vấn đề đối kháng thích nghi).
- **Không gian:** $O(\log N)$ vòng $\times O(\log^2 N)$ bit mỗi sketch $\times N$ đỉnh $= O(N \log^3 N)$ bit.

Bài toán ước lượng moment

Bài toán (F_p moment estimation)

Cho x cập nhật trong turnstile, ước lượng $\|x\|_p^p = \sum_i |x_i|^p$ với sai số nhân $(1 \pm \varepsilon)$.

- $p = 0$: số phần tử phân biệt (đã xét).
- $p = 1$: $\|x\|_1$; trong turnstile chặt chỉ cần một bộ đếm, nhưng turnstile tổng quát thì không tầm thường.
- $p = 2$: $\|x\|_2^2$ — “self-join size” trong cơ sở dữ liệu; đây là trường hợp đẹp nhất.
- $p = \infty$: $\max_i |x_i|$ — đòi hỏi $\Omega(n)$ bit (quy dẫn từ DISJOINTNESS).

Phân kỳ pha

Với $0 < p \leq 2$, không gian $\text{poly}(\varepsilon^{-1} \log n)$ là đủ. Với $p > 2$, cần $\Theta(n^{1-2/p})$ tới thừa số $\log$.

Sketch AMS cho F_2

AMS (Alon–Matias–Szegedy, 1996)

Chọn $\sigma_1, \dots, \sigma_n \in \{-1, 1\}$ từ họ độc lập *bậc bốn* và duy trì

$$Y = \sum_{i=1}^n \sigma_i x_i.$$

Ước lượng $\|x\|_2^2$ bằng Y^2 .

- **Không chệch:** $\mathbb{E}Y^2 = \sum_{i,j} \mathbb{E}[\sigma_i \sigma_j] x_i x_j = \sum_i x_i^2 = \|x\|_2^2$ (cần độc lập bậc hai).

- **Phương sai:** khai triển $\mathbb{E}Y^4$ cần độc lập *bậc bốn*; kết quả

$$\text{Var}[Y^2] = \mathbb{E}Y^4 - (\mathbb{E}Y^2)^2 \leq 2 \|x\|_2^4.$$

- Trung bình $s = O(1/\varepsilon^2)$ bản sao rồi lấy trung vị $O(\log(1/\delta))$ khối.

Định lý

Có sketch tuyến tính ước lượng $\|x\|_2^2$ với sai số $(1 \pm \varepsilon)$, xác suất $1 - \delta$, dùng $O(\varepsilon^{-2} \log(1/\delta) \log n)$ bit.

Tương đương: $\Pi \in \mathbb{R}^{m \times n}$ ma trận dấu ngẫu nhiên, $m = O(\varepsilon^{-2} \log(1/\delta))$, ước lượng $\|\Pi x\|_2^2 / m$ — đây chính là biến đổi Johnson–Lindenstrauss.

Sketch p -ổn định của Indyk

Định nghĩa (Phân phối p -ổn định)

Phân phối $\mathcal{D}_p$ là p -ổn định nếu với $Z_1, \dots, Z_n$ độc lập cùng phân phối $\mathcal{D}_p$ và $x \in \mathbb{R}^n$ bất kỳ,

$$\sum_{i=1}^n x_i Z_i \sim \|x\|_p \cdot Z, \quad Z \sim \mathcal{D}_p.$$

- Tồn tại với mọi $p \in (0, 2]$. Trường hợp $p = 2$: phân phối Gauss; $p = 1$: phân phối Cauchy.
- **Thuật toán:** lấy $\Pi \in \mathbb{R}^{m \times n}$ với các phần tử độc lập cùng phân phối $\mathcal{D}_p$, duy trì $y = \Pi x$. Khi đó mỗi $y_r \sim \|x\|_p Z_r$.
- **Ước lượng:** $\widetilde{\|x\|_p} = \frac{\text{median}_r |y_r|}{\text{median } |\mathcal{D}_p|}$, với $m = O(\varepsilon^{-2} \log(1/\delta))$.
- Vì sao trung vị mà không phải trung bình? Với $p < 2$, phân phối p -ổn định có đuôi nặng — Cauchy thậm chí không có kỳ vọng hữu hạn. Trung vị là thống kê bền vững và tập trung tốt.

Vấn đề ngẫu nhiên: chương trình rẽ nhánh và PRG

Khó khăn

Sketch p -ổn định cần mn biến ngẫu nhiên độc lập — lưu Π tốn nhiều bộ nhớ hơn cả việc lưu x ! Ngược lại, với AMS ta chỉ cần độc lập bậc bốn nên hạt giống $O(\log n)$ bit là đủ; với $\mathcal{D}_p$ không có mẹo tương tự.

- **Ý tưởng của Indyk:** xem toàn bộ thuật toán streaming như một *chương trình rẽ nhánh* (branching program) độ rộng 2^S và độ dài R , đọc các bit ngẫu nhiên theo thứ tự.
- **Bộ sinh giả ngẫu nhiên Nisan** đánh lừa được mọi chương trình rẽ nhánh như vậy, với hạt giống độ dài $O(S \log R)$ bit.
- Kết quả: thuật toán chỉ cần lưu hạt giống và *tính lại* các phần tử của Π khi cần, cho không gian tổng cộng

$$O(\varepsilon^{-2} \log n \log(1/\delta)) \text{ bit.}$$

- Đây là một khuôn mẫu chung: *phi ngẫu nhiên hoá* (derandomization) các sketch bằng PRG cho bộ nhớ nhỏ.

Chặn dưới: bức tranh đối ngẫu

Mọi chặn trên ở trên đều đi kèm câu hỏi “có tối ưu không?”. Hai kỹ thuật chủ đạo:

- **Lập luận nén** (compression arguments): nếu tồn tại thuật toán dùng S bit thì tồn tại đơn ánh $\text{Enc} : \mathcal{A} \rightarrow \{0, 1\}^{g(S)}$, suy ra $g(S) \geq \log_2 |\mathcal{A}|$. Thường dùng kèm mã sửa sai để dựng $\mathcal{A}$ lớn.
 - F_0 chính xác và tất định: $\Omega(n)$ bit.
 - F_0 xấp xỉ nhưng tất định: $\Omega(n)$ bit.
 - F_0 chính xác nhưng ngẫu nhiên: $\Omega(n)$ bit.

Kết luận: *cả xấp xỉ lẫn* ngẫu nhiên hoá đều không thể bỏ.

- **Độ phức tạp truyền thông**: quy dẫn từ EQUALITY, DISJOINTNESS, INDEX, GAPHAMMING.
 - $\text{INDEX} \rightarrow \text{GAPHAMMING} \rightarrow F_0$ cho $\Omega(1/\varepsilon^2)$ bit.
 - DISJOINTNESS nhiều người chơi cho $\Omega(n^{1-2/p})$ với $F_p, p > 2$.

Bảng tổng kết

Bài toán	Thuật toán	Không gian	Mô hình
Đếm xấp xỉ	Morris++	$O(\varepsilon^{-2} \log \frac{1}{\delta} \log \log n)$ bit	chỉ chèn
F_0	KMV	$O(\varepsilon^{-2} \log n)$ bit	chỉ chèn
F_0	lấy mẫu hình học	$O(\varepsilon^{-2} \log^2 n)$ bit	chỉ chèn
Phân vị	q-digest	$O(\varepsilon^{-1} \log U)$ từ	chỉ chèn, vũ trụ hữu hạn
Phân vị	MRL	$O(\varepsilon^{-1} \log^2(\varepsilon n))$ từ	so sánh, tất định
Phân vị	KLL	$O(\varepsilon^{-1} \log \log \frac{1}{\delta})$ từ	so sánh, ngẫu nhiên
ℓ_1 heavy hitters	CountMin	$O(k \log \frac{1}{\delta})$ từ	turnstile
ℓ_2 heavy hitters	CountSketch	$O(k \log \frac{1}{\delta})$ từ	turnstile
Liên thông đồ thị	AGM	$O(N \log^3 N)$ bit	turnstile chặt
F_2	AMS	$O(\varepsilon^{-2} \log \frac{1}{\delta} \log n)$ bit	turnstile
$F_p, p \in (0, 2]$	Indyk p -ổn định	$O(\varepsilon^{-2} \log \frac{1}{\delta} \log n)$ bit	turnstile

Những ý tưởng cần nhớ

- 1 **Ước lượng không chệch + phương sai chặn được**, rồi *trung vị của trung bình* để đạt $(1 \pm \varepsilon)$ với xác suất $1 - \delta$.
- 2 **Độc lập hạn chế**: độc lập bậc hai đủ cho Chebyshev; bậc bốn đủ cho phương sai của Y^2 . Hạt giống chỉ $O(k \log n)$ bit thay vì $n \log n$.
- 3 **Tính tuyến tính là tất cả**: sketch tuyến tính cho ngay mô hình turnstile, tính hợp nhất, và tính toán phân tán. Sketch của một siêu đỉnh = tổng sketch các đỉnh (AGM).
- 4 **Lấy mẫu hình học**: chia luồng thành $\log n$ mức với mật độ giảm dần; luôn có một mức “vừa đúng”.
- 5 **Dấu ngẫu nhiên**: biến nhiễu thiên lệch (CountMin) thành nhiễu triệt tiêu (CountSketch), đổi ℓ_1 lấy ℓ_2 .
- 6 **Cấu trúc cây**: cây dyadic để tăng tốc truy vấn; cây nhị phân cho q-digest; chuỗi bộ nén cho MRL/KLL.
- 7 **Chặn dưới định hướng thiết kế**: biết bài toán khó ở đâu để không phí công.

Tài liệu tham khảo

- J. Nelson. *Sketching Algorithms*. Ghi chú bài giảng, 2020.
<https://www.sketchingbigdata.org/fall20/lec/notes.pdf>
- R. Morris. Counting large numbers of events in small registers. *CACM*, 1978.
- P. Flajolet, G. N. Martin. Probabilistic counting algorithms for data base applications. *JCSS*, 1985.
- Z. Bar-Yossef và cộng sự. Counting distinct elements in a data stream. *RANDOM*, 2002.
- N. Alon, Y. Matias, M. Szegedy. The space complexity of approximating the frequency moments. *STOC*, 1996.
- G. Cormode, S. Muthukrishnan. An improved data stream summary: the count-min sketch. *J. Algorithms*, 2005.
- M. Charikar, K. Chen, M. Farach-Colton. Finding frequent items in data streams. *ICALP*, 2004.
- Z. Karnin, K. Lang, E. Liberty. Optimal quantile approximation in streams. *FOCS*, 2016.
- K. J. Ahn, S. Guha, A. McGregor. Analyzing graph structure via linear measurements. *SODA*, 2012.
- P. Indyk. Stable distributions, pseudorandom generators, embeddings, and data stream computation. *JACM*, 2006.
- G. Cormode, K. Yi. *Small Summaries for Big Data*. Cambridge University Press, 2020.

Cảm ơn đã theo dõi.

Câu hỏi và thảo luận.